

现代 C++ 特性初探

@ 102400233 张子良

doubeecat@qq.com

课件中的笔记: [GitHub 库](#)

CONTENT

今天聊什么？

- 1 流(streams) ↗
- 2 自动推导类型(auto) ↗
- 3 基于范围的遍历 ↗
- 4 constexpr ↗
- 5 lambda 函数 ↗

1. 流(streams)

流的存在？

与 console 和 files 等等对象进行交互的方式。很酷的在于流提供了一个通用接口。

举个例子，`cout` 如何输出一个东西到控制台？程序中的数据对象 ^{类型转化} $\rightarrow$ `string` 表示 ^{输出} $\rightarrow$ 控制台，第二部分就是流所做的事情了。

stringstream

stringstream 分为两种，有 ostringstream 和 istream，代表的是输出流和输入流。ostringstream 可以使用 << 运算符向里面写入东西。也就相当于一个缓冲区。

我们可以使用 .str() 把 ostringstream 中的东西转换为字符串。

```
ostringstream oss("it's me");  
cout << oss.str() << endl;  
//it's me  
oss << "888hi!";  
cout << oss.str();  
//888hi!e
```

想象一下我们有一个字符指针一直在移动，最开始构造这个流的时候，指针从起点开始，所以我们覆盖了缓冲区。如果我们想让字符指针从流末尾开始，我们可以使用 `stringstream::ate` 构造一个流：

```
ostringstream oss("it's me",stringstream::ate);  
cout << oss.str() << endl;  
//it's me  
oss << "888hi!";  
cout << oss.str();  
//it's me888hi!
```

`stream` 比 `buffer` 更厉害的在于，你可以直接从 `stream` 中读入数字或者其他什么东西。比如我们可以把读入流中的 `string` 也自动变为 `int` 等类型，并且以符号作为分割，特别智能：

```
istringstream iss("114 yuan");  
int price; string unit;  
iss >> price;  
iss >> unit;  
cout << price << " " << unit << endl;  
//57 yuan
```

`stringstream` 优秀之处在于，他把一个字符串给划分成了若干个 tokens，比如：

```
16.9 \n Ounces. . \t\n\n -38271
```

`16.9`, `Ounces`, `.`, `-38271` 就是我们分出来的四个 tokens. 但是 tokens 的分割也是基于我们如何读入的，比如上面这段，如果我们用 `int` 类型的变量去读入，那么第一个 token 就是 `16`.

我们对于每个 `ostringstream` 的 IO，分为两步：

1. 把对应变量转换为字符串
2. 插入到对应的 `ostringstream` 的缓冲区。

`istringstream` 也是类似：

1. 从缓冲区中读取字符串
2. 把对应字符串转化为所求类型的变量

stringstream 支持链式调用，也就是可以实现 iss >> var1 >> var2 >> ... 的使用。

刚才我们提到了流的字符指针，我们同样可以通过一些函数来知道当前指针的位置，这些东西比较底层：

功能	istringstream	ostringstream
获取指针位置	oss.tellp()	iss.tellg()
改变指针位置	oss.seekp(pos)	iss.seekg(pos)
获取偏移量	streamoff()	

```
ostringstream iss("114 yuan",stringstream::ate);
cout << iss.tellp();
//8 这里设置了 stringstream::ate, 所以是在末尾
ostringstream iss1("114 yuan");
cout << iss1.tellp();
//0
```

我们可以实现一个简单的函数，实现从字符串到数字的转换啦！

```
int StringToInteger(const string &s) {  
    int number;  
    istringstream iss(s);  
    iss >> number;  
    return number;  
}
```

这个函数实现了从字符串内读入了一个数的功能，我们也可以实现从字符串自动读入一堆数，然后返回一个 `vector` 的功能：

```
vector<int> StringToVector(const string &s) {  
    vector<int> number;  
    istringstream iss(s);  
    int x;  
    while (iss >> x) {  
        number.push_back(x);  
    }  
    return number;  
}  
  
int main() {  
    vector<int> vec;  
    vec = StringToVector("1 2 3 4 5");  
    for (auto x : vec) cout << x*x << "\n";  
    return 0;  
}  
  
//1 4 9 16 25
```

状态位

反应流到底发生什么了，分成四种：

- Good bit: 准备好读入/输出了
- Fail bit: 先前操作失败，冻结此后操作
- EOF bit: 先前操作已经到达了缓冲区内容末尾
- Bad bit: 由于流缓冲区上的输入/输出操作失败而导致的错误。（比如你文件被删掉了）

状态位之间并不矛盾，比如 G 和 B 通常同时出现在类型不匹配的时候，正常读入到 EOF 也会同时出现 G 和 E. 良好和失败并不对立，正如考试一样^_^

状态位的存在帮助我们很好地去检查我们在使用流时发生的错误。

标准 `iostream`

以下四个流都是与 console 交互的：

- `cin`：标准输入流
- `cout`：标准输出流（buffered）
- `cerr`：标准错误流（unbuffered）
- `clog`：标准错误流（buffered）

`cin, cout` 和控制台交互的原则和我们在 `stringstream` 大同小异，但是特点在于，如果缓冲区这时候没有东西，`cin` 会等待用户输入。提取器会帮助我们跳过前导空格，但是不会消耗下一个非空格字符。

注意，`cin` 如果读入失败了，这个时候 Fail bit 为 1，此后所有 `cin` 操作都倒闭了。

我们也可以使用 `getline()` 代替运算符，这样我们就可以忽略空格读入一整行了，`getline` 是以换行符为分界，每次消耗掉一个换行符。主要作用在于确保我们可以清空缓存区，不会让我们读入变得差。使用方法也很简单：

```
cin >> str;  
//更改成  
getline(cin, str);
```

`getline` 函数返回的是 `bool`，指示我们的操作是否成功。基于此我们可以写出一个新的更加完备的 `getInteger()`

```
int getInterget() {
    while (1) {
        string line;
        if (!getline(cin,line))
            throw domain_error();
        istringstream iss(line);
        int res;char trash;
        if (iss >> res && !(iss >> trash))
            return res;
    }
}
```

对于与文件交互，我们使用 `ifstream` 和 `ofstream`

```
int var1;  
ifstream in("in.txt",ifstream::in);  
ofstream out("out.txt",ofstream::out);  
in >> var1;  
out << var1;
```

这段代码实现了从 `in.txt` 中读入，并且输出到 `out.txt` 中。因为对于 `fstream` 的使用本质上和我们上面用 `stream` 没有区别，所以就直接省略了。

2. 自动推导类型(auto)

来点 auto 笑话

听说这是 C++50 标准（狗头）

```
// it can't compile in any compiler at present.  
// but if in 2050...? who knows ^_^  
#auto <auto>  
auto auto {  
    auto auto() -> auto{};  
    return auto;  
}
```

auto

`auto` 关键字自从 C++11 标准中被引入。这个关键字可以指示编译器自动推导类型，或者直接推导表达式。

在大部分时候，其实是推荐使用 `auto` 关键字的，它有以下好处：

- **可靠性**：如果表达式的类型发生更改（包括函数返回类型发生更改的情况），它也能工作。
- **性能**：确保不会进行转换。
- **可用性**：不必担心类型名称拼写困难和拼写有误。
- **效率**：代码会变得更高效。

但是在一些情况，比如你需要一个特殊类型的变量（譬如使用 `double` 才可以，使用 `float` 会丢失精度）之下就不能使用 `auto` 了。

auto

使用 `auto` 的过程和使用任意一个数据类型是一样的。但是它本身不是一个数据类型，而是一个占位符。所以它不能用于强制转换 `or` 运算符，例如 `sizeof` 之中。并且使用 `auto` 时候会自动删除 `const,&` 限定符，所以你需要自己声明是否引用。

```
#include <iostream>
using namespace std;
int main( )
{
    int count = 10;
    int& countRef = count;
    auto myAuto = countRef;
    countRef = 11;
    cout << count << " ";
    myAuto = 12;
    cout << count << endl;
}
```

这里的 `myAuto` 就被解析为一个 `int` 类型的变量了。

auto samples

接下来我们给大家展示一些用 `auto` 简化代码的实际案例：

`iterator` 太长不想打？ `auto` 启动！

```
std::set<int> s;  
auto it = s.begin();  
// equals to  
std::set<int>::iterator it = s.begin();
```

下面的代码片段将变量 `x` 初始化为类型 `int`，将变量 `y` 初始化为对类型 `const int` 的引用，将变量 `fp` 初始化为指向返回类型 `int` 的函数的指针。

```
int f(int x) { return x; }
int main()
{
    auto x = f(0);
    const auto& y = f(1);
    int (*p)(int x);
    p = f;
    auto fp = p;
    //...
}
```

3. 基于范围的遍历

基于范围的遍历

对一个 `vector` 执行从头到尾的遍历，以前我们可能会怎么做？

```
std::vector<int> vec;  
size_t n = vec.size()  
for (int i = 0; i < n; ++i) {  
    cout << vec[i] << endl;  
}
```

但是我们其实可以：

```
std::vector<int> vec;  
for (int x : vec) {  
    cout << x << endl;  
}
```


基于范围的遍历

基于范围的遍历的基本语法：

```
for (declaration : expression) {}
```

我们推荐对于有 `.begin() .end()` 的迭代器再使用这样的遍历。并且我们推荐直接使用 `auto` 推导类型。同样地，我们可以使用 `const` , `&` 这样的 来进行修饰。例如：

```
int val[8] = {3,1,4,1,5,9,2,6};  
for (const auto &x : val) {  
    cout << x << endl;  
}
```

4. constexpr

constexpr

C++11 中引入了 `constexpr` 关键字。`constexpr` 是 `const expression` 的缩写，即常量表达式。

常量表达式是指值不会改变且编译期可以得到结果的表达式。特点在于：

- 值不会改变（这一点和普通 `const` 一样）
- 编译期就能得到结果！（普通 `const` 不一定保证）

常见的常量表达式

- 字面值（如 42）
- 用常量表达式初始化的 `const` 对象
- 一个对象（或表达式）是否是常量表达式取决于类型和初始值，如：

```
int i1 = 42;           // i1 不是常量表达式：初始值 42 是字面值，但 i1 不是 const 类型
const int i2 = i1;     // i2 不是常量表达式：初始值 i1 不是常量表达式
const int i3 = 42;     // i3 是常量表达式：用字面值 42 初始化的 const 对象
const int i4 = i3 + 1; // i4 是常量表达式：用常量表达式 i3 + 1 初始化的 const 对象
const int i5 = getValue(); // 如果 getValue() 是普通函数，则 i5 值要到运行时才能确定，则不是常量表达式
```

但是，我们用的 `constexpr` 与之不同：

constexpr 变量

上面的例子可以看出，不能直接判断一个 `const` 对象是否是常量表达式：例如 `i4` 是否是常量表达式取决于 `i3` 是否是常量表达式，而 `i4` 又可能用来初始化其他常量表达式。在复杂的系统中，很难一眼看出某个 `const` 对象是否是常量表达式。

C++11 允许把变量声明为 `constexpr` 类型，此时编译器会保证 `constexpr` 变量是常量表达式（否则编译报错）。换句话说，只要看到 `constexpr` 类型的变量，则一定能够在编译期取得结果，可以用在需要常量表达式的场景。

```
int i1 = 42;
constexpr int i2 = i1; // constexpr 变量 'i2' 必须由常量表达式初始化。不允许在常量表达式中读取非 const 变量 'i1'
constexpr int i3 = 42; // i3 是常量表达式
constexpr int i4 = i3 + 1; // i4 是常量表达式
constexpr int i5 = getValue(); // 只有 getValue() 是 constexpr 函数时才可以，否则编译报错
```

constexpr 函数

constexpr 函数是指能用于常量表达式的函数。

需要强调的是，constexpr 函数既能用于要求常量表达式/编译期常量的语境，也可以作为普通函数使用。

注意：constexpr 函数不一定返回常量表达式！

只有 constexpr 的所有实参都是常量表达式/编译期常量时，constexpr 函数的结果才是常量表达式/编译期常量。只要有一个参数在编译期未知，那就和普通函数一样，在运行时计算。

“ 为了保证 constexpr 函数在编译时能随时展开计算，constexpr 函数隐式内联。内联函数和 constexpr 函数不同于其他函数，允许定义多次，但要保证所有的定义一致。正因如此，内联函数和 constexpr 函数一般定义在头文件中。

```
constexpr int sum(int a, int b) {  
    return a + b;  
}
```

```
constexpr int i1 = 42;  
constexpr int i2 = sum(i1, 52); // 所有参数都是常量表达式，sum 的结果也是常量表达式，在编译期求值
```

```
int AddThree(int i) {  
    return sum(i, 3); // i 不是常量表达式，此时 sum 作为普通函数使用  
}
```

使用 constexpr 的好处

编译器可以保证 constexpr 对象是常量表达式（能够在编译期取得结果），而 const 对象不能保证。如果一个 const 变量能够在编译期求值，将其改为 constexpr 能够让代码更清晰易读

constexpr 函数可以把运行期计算迁移至编译期，使得程序运行更快（但会增加编译时间）

对于常量表达式（编译期值已知），编译器可以进行更多优化，比如放到只读内存中。但这并不是 constexpr 特有的，有的 const 变量也是常量表达式

In conclusion...

修饰对象的时候，可以把 `constexpr` 当作加强版的 `const`： `const` 对象只表明值不会改变，不一定能够在编译期取得结果；

`constexpr` 对象不仅值不会改变，而且保证能够在编译期取得结果

`constexpr` 函数既可以用于编译期计算，也可以作为普通函数在运行期使用

5. Lambda函数

Lambda 函数

~~喜欢 Python 的小朋友你们好啊，我是 Lambda.~~

如果有学过 Python 的同学可能会对这个概念很熟悉，对的朋友！Lambda 是在 C++11 标准被引入的一个新概念。其本身也是实现函数式编程的一个重要概念。它允许我们在任意地方定义函数，并且在接下来的函数中直接调用。

Lambda 函数的关键概念在于捕获列表，参数列表，返回类型和函数体。其后三者与普通函数相同，关键在于捕获列表是什么？

捕获列表

我们可以先看一个简单的 Lambda 函数实例：

```
auto f = [](int x) -> int {  
    return x + 1;  
}
```

大家可以看到，相比于正常的函数，我们多了一个 `[]` 和 `->`。而 `[]` 就代表捕获列表。用于指定 Lambda 表达式可以访问的外部变量，以及是按值还是按引用的方式访问。例如**值捕获**(capture by value)就是：

```
int x = 10;  
auto add[x](int y) -> int {  
    return x + y;  
}  
cout << add(5);
```

捕获相当于把对应的值拷贝到表达式中，作为一个成员可以直接用。值捕获是无法修改值的。

捕获列表

引用捕获（capture by reference）：在捕获列表中使用 & 加变量名，表示将该变量的引用传递到 Lambda 表达式中。

作为一个数据成员。引用捕获的变量在 Lambda 表达式调用时才确定，会随着外部变量的变化而变化。引用捕获的变量可以在 Lambda 表达式中修改。例如：

```
int x = 10;
auto add[&x](int y) -> int {
    return x + y;
}
x = 20;
cout << add(5);
//输出 25
```

Lambda表达式的优点

Lambda表达式相比于普通函数和普通类，有以下几个优点：

- 简洁：Lambda表达式可以省略函数名和类名，直接定义和使用，使得代码更加简洁和清晰。
- 灵活：Lambda表达式可以捕获外部变量，可以作为函数参数，也可以作为函数返回值，使得代码更加灵活和方便。
- 安全：Lambda表达式可以控制外部变量的访问方式，可以避免全局变量的定义，可以避免悬空指针和无效引用的产生，使得代码更加安全和稳定。

Lambda 表达式的实例

主播主播，你说的 Lambda 确实很强，那有没有什么更强势的应用呢？

有的有的，大家可以看一下下面这个东西：

```
struct point {
    int x,y;
};

int main() {
    vector<point> vec;
    vec.emplace_back(1,2);vec.emplace_back(5,6);vec.emplace_back(3,4);
    sort(vec.begin(),vec.end(),
    [](const point &p1,const point& p2) {
        return p1.x > p2.x;
    });
    for (auto [x,y] : vec) {
        cout << x << "," << y << endl;
    }
}
```

泛型 Lambda

细心的同学应该已经注意到了，我刚才用 `auto` 关键字来修饰的 Lambda 那这是什么意思呢？

C++14 允许在 Lambda 表达式的参数列表和返回值类型中使用 `auto` 关键字，从而实现泛型 Lambda，即可以接受任意类型的参数和返回任意类型的值的 Lambda 表达式。例如：

```
auto f = [] (auto x) -> auto
{
    if (is_integral<decltype(x)>::value) // 如果 x 是整数类型
    {
        return x * 2; // 返回 x 的两倍
    }
    else if (is_floating_point<decltype(x)>::value) // 如果 x 是浮点类型
    {
        return x / 2; // 返回 x 的一半
    }
    else // 其他类型
    {
        return x; // 返回 x 本身
    }
};
```


In conclusion...

作为一个 C++11 引入的语法糖（probably 个人最喜欢的语法糖），Lambda 让我们的代码变得更加清晰了。喜欢捏。

欢迎交流 ~



邮箱: doubeecat@qq.com



QQ: 1359446223



Github: Doubeecat